МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра Название кафедры

ОТЧЕТ

по практической работе № 1
по дисциплине «Системы параллельной обработки сигналов»

Тема: «Запуск параллельной программы на различном числе одновременно работающих процессов. Упорядочение вывода результатов»

Студент гр. 0000 _____ Фамилия И. О.
подпись

Преподаватель _____ Фамилия И. О.
подпись

Санкт-Петербург
2026

1 Цель работы

Познакомиться с запуском MPI-программы на различном числе процессов, обменом сообщениями между процессами и способами управления порядком вывода результатов параллельного выполнения.

2 Формулировка задания

Отдельный номер варианта в исходном задании не задан; работа выполнена по единственной предложенной формулировке.

Задание 1. Все процессы, кроме процесса с рангом 0, передают значение своего ранга нулевому процессу. Процесс с рангом 0 сначала печатает значение собственного ранга, затем принимает сообщения с рангами остальных процессов и также выводит их. Порядок приёма сообщений заранее не определён и может изменяться от запуска к запуску.

Необходимо несколько раз запустить программу при фиксированном числе процессов (например, при восьми), проанализировать порядок вывода сообщений и сформулировать правило, которое его определяет.

Задание 2. Модифицировать программу так, чтобы порядок вывода сообщений соответствовал номерам процессов. Сравнить результаты работы двух вариантов программы.

3 Описание алгоритмов

Реализован вариант задания 1 на языке C с MPI. Все p процессов вызывают MPI_Init, определяют ранг и число процессов. Каждый процесс с рангом $i > 0$ отправляет одно целое число i процессу 0 с тегом 0. Процесс 0 печатает собственный ранг и выполняет ровно $p - 1$ операций MPI_Recv с источником MPI_ANY_SOURCE, печатая полученное значение и фактического отправителя из MPI_Status. Все процессы завершают работу через MPI_Finalize.

При проверке исправлена граница цикла: ранее выполнялось p приёмов, хотя отправлялось только $p - 1$ сообщений. Лишний приём приводил к зависанию. При $p = 1$ исправленная программа только печатает ранг 0 и завершается.

Для задания 2 создана отдельная программа. В цикле по $i = 1, \dots, p - 1$ источником приёма указывается процесс i вместо MPI_ANY_SOURCE. Все отправители работают параллельно. Процесс 0 по очереди ожидает сообщения от процессов 1, 2, …, $p - 1$ и тем самым задаёт порядок печати.

4 Формальное описание с помощью сетей Петри

Используется параметризованная сеть Петри с цветными фишками. Формализация выполнена с опорой на основные положения аппарата сетей Петри [1]. Для каждого $i \in \{1, \dots, p-1\}$ место A_i содержит одну начальную фишку готовности отправителя, B_i обозначает завершение его обмена. Место R содержит фишку состояния принимающего процесса (k, L): число принятых сообщений и последовательность уже напечатанных рангов. Начальное место Z содержит одну фишку готовности процесса 0. Место F обозначает завершение приёма.

Переход $t_0 : Z \to R$ печатает 0 и создаёт фишку $(0, [0])$. Для каждого $i > 0$ переход обмена имеет входы A_i, R и выходы B_i, R:

$$A_i + R(k, L) \xrightarrow{t_i} B_i + R(k+1, L \parallel [i]). \tag{1}$$

Выражение (1) задаёт функционал t_i: передать ранг i, принять его процессом 0 и напечатать. Обмен отправителя с получателем здесь моделируется одним совместным переходом; внутренняя буферизация MPI абстрагирована. В неупорядоченном варианте допустим любой ещё не сработавший t_i. В упорядоченном варианте дополнительное условие перехода: $i = k + 1$. Переход $t_f : R \to F$ разрешён только при $k = p - 1$; он завершает фазу приёма.

Каждый t_i срабатывает ровно один раз, поскольку фишка A_i имеется в одном экземпляре. Фишка R задаёт последовательную печать процессом 0. Итоговый список содержит все ранги ровно по одному разу. При $p = 1$ после t_0 сразу разрешён t_f.

Графическая схема параметризованной сети представлена на рисунке 1.

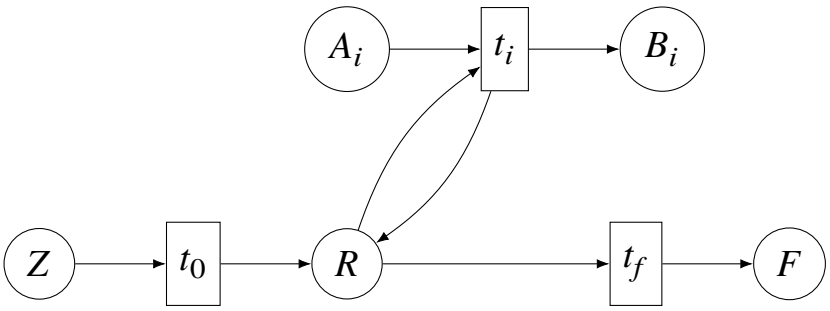


Рисунок 1 – Параметризованная сеть Петри для обмена сообщениями

Графическая схема на рисунке 1 показывает один из $p - 1$ однотипных каналов отправителей. Дуги между R и t_i отражают изменение состояния принимающего процесса после каждого сообщения.

5 Реализация и функциональная проверка

Полный листинг программы с неупорядоченным приёмом приведён в приложении А, листинг программы с упорядоченным приёмом --- в приложении Б.

5.1. Ход выполнения задания 1

Программа собрана средствами CMake. Цель task1_unordered компилируется как программа на языке C11 и связывается с библиотекой MPI. Для компиляторов GNU и Clang включены предупреждения -Wall, -Wextra и -Wpedantic. Сборка завершилась без ошибок.

После инициализации MPI каждый процесс определяет свой ранг. Процессы с рангами больше нуля передают свои номера процессу 0. Нулевой процесс сначала выводит 0, затем выполняет $p - 1$ операций MPI_Recv. В каждой операции источник задан константой MPI_ANY_SOURCE, поэтому может быть принято сообщение от любого ещё не обслуженного отправителя.

Для начальной проверки программа запущена на одном и двух процессах. При $p = 1$ отправители отсутствуют, поэтому выводится только 0. При $p = 2$ процесс 0 получает единственное сообщение от процесса 1. Результат сборки и запусков представлен на рисунке 2.

```
❯ cmake --build build
  mpirun -np 1 ./build/task1_unordered
  mpirun -np 2 ./build/task1_unordered
[100%] Built target task1_unordered
rank: 0
rank: 0
Received rank 1, Sender 1
```

Рисунок 2 – Сборка и запуск неупорядоченной программы на одном и двух процессах

Затем выполнено пять запусков при $p = 8$. В каждом опыте программа завершилась, нулевой ранг был напечатан первым, а каждый из рангов 1--7 встретился ровно один раз. Полученные последовательности приведены в таблице 1.

Таблица 1 – Последовательности рангов в пяти запусках на восьми процессах

Запуск	Ранги в порядке печати
1	0, 7, 3, 5, 2, 6, 4, 1
2	0, 2, 6, 7, 3, 4, 1, 5
3	0, 1, 2, 3, 4, 5, 6, 7
4	0, 1, 4, 2, 7, 3, 6, 5
5	0, 7, 1, 3, 4, 5, 6, 2

Первым всегда печатается 0. Далее вывод соответствует порядку, в котором операции приёма выбирают подходящие сообщения. Источник MPI_ANY_SOURCE допускает приём от любого процесса, поэтому порядок отправителей не задаётся. На результат влияют планирование процессов и работа MPI; нельзя гарантировать ни возрастание рангов, ни порядок начала отправок, ни строгую сортировку по времени физического прихода сообщений. Одинаковые последовательности при повторных запусках тоже допустимы. Передаются сообщения с целыми числами, а не сигналы ОС. Полный вывод пяти запусков показан на рисунке 3.

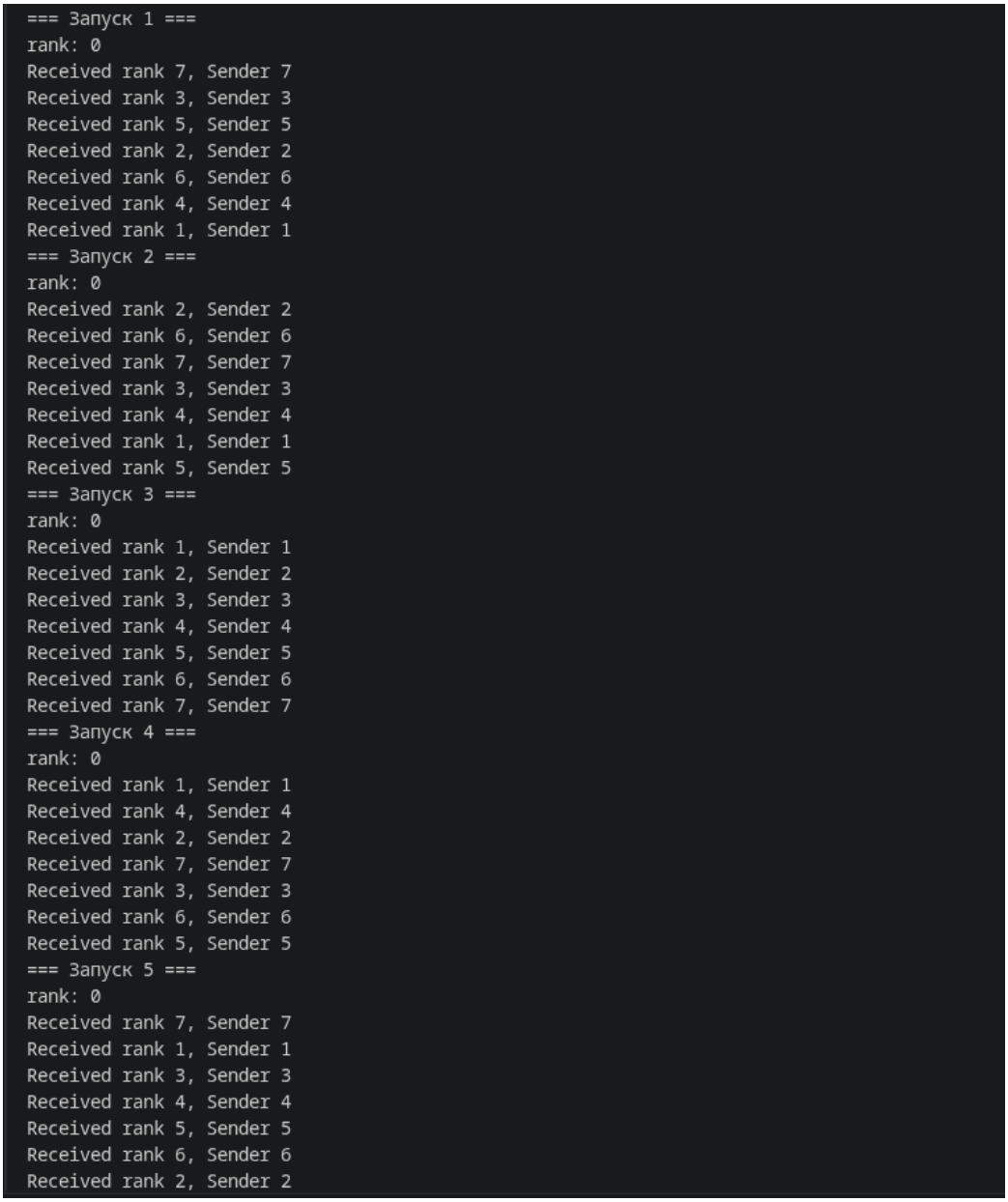


Рисунок 3 – Пять запусков неупорядоченной программы на восьми процессах

5.2. Ход выполнения задания 2

Вторая программа получена изменением источника в операции MPI_Recv: вместо произвольного отправителя MPI_ANY_SOURCE используется значение счётчика цикла i. Алгоритм отправителей и количество операций приёма остались без изменений. Таким образом, на первой итерации процесс 0 ожидает сообщение от процесса 1, на второй --- от процесса 2 и далее до процесса $p - 1$. Сообщение от процесса с большим рангом может

быть готово раньше, но будет обработано только после всех отправителей с меньшими рангами.

Программа собрана средствами CMake и сначала проверена при $p = 1, 2, 4$. При одном процессе выводится только 0; при двух и четырёх процессах получены строго возрастающие последовательности рангов (рисунок 4).

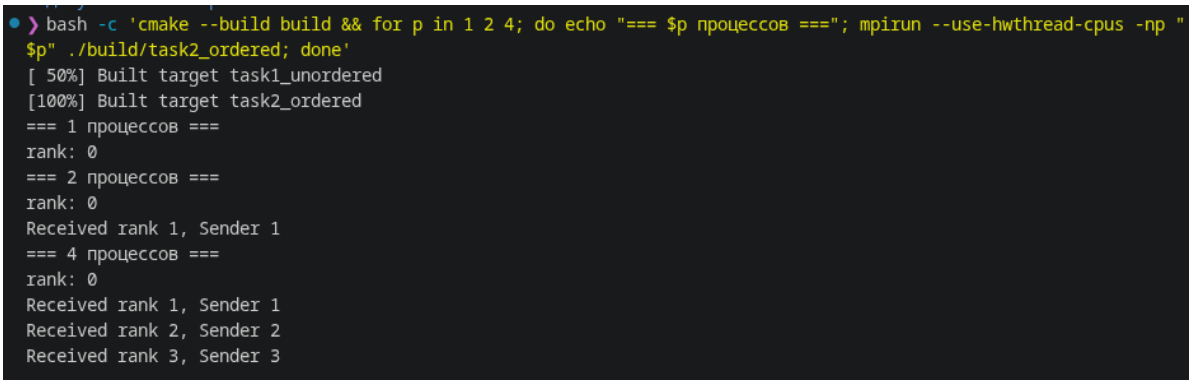


Рисунок 4 – Сборка и запуск упорядоченной программы на одном, двух и четырёх процессах

Для проверки устойчивости результата выполнены три запуска при $p = 8$. Во всех запусках получена одна последовательность $0, 1, 2, 3, 4, 5, 6, 7$, что подтверждает заданный программой порядок приёма. Результаты повторных запусков представлены на рисунке 5.

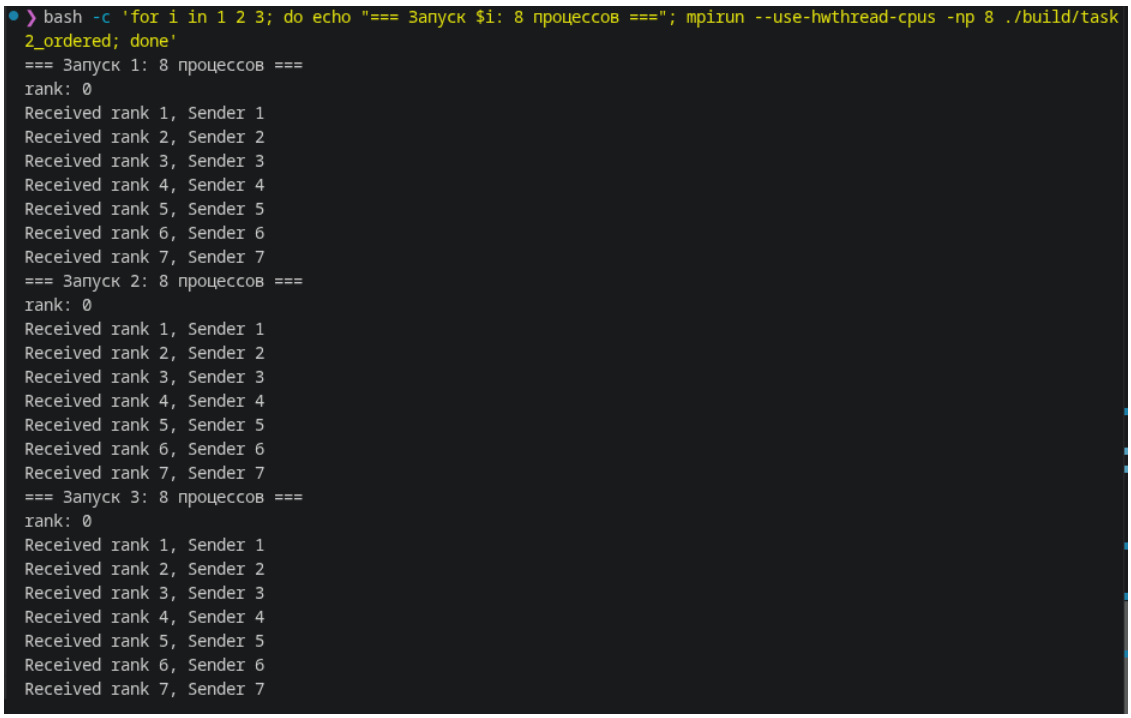


Рисунок 5 – Три запуска упорядоченной программы на восьми процессах

Дополнительно выполнен функциональный запуск при $p = 16$. Поскольку используемый процессор предоставляет 12 аппаратных потоков, применён параметр --oversubscribe. Этот опыт подтверждает корректность порядка $0, 1, \ldots, 15$, но из-за переподписки не используется для оценки производительности. Результат показан на рисунке 6.

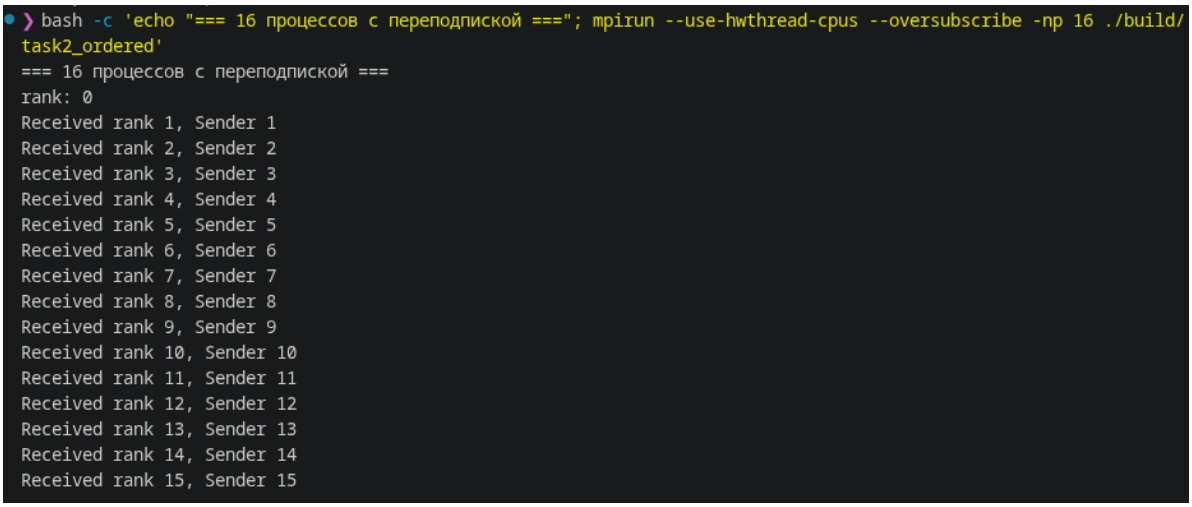


Рисунок 6 – Запуск упорядоченной программы на шестнадцати процессах с переподпиской

6 Экспериментальное исследование

6.1. Методика измерения

Для неупорядоченного алгоритма создана экспериментальная программа, листинг которой приведён в приложении В. Параметр n задаёт число целых чисел в сообщении каждого отправителя. Исследованы размеры $n = 1$, 1024 и 65536, что соответствует 4 байтам, 4 Кбайт и 256 Кбайтам.

Заполнение буфера выполнялось до начала замера. Все процессы синхронизировались вызовом MPI_Barrier, после чего на каждом процессе фиксировалось начало и окончание обмена функцией MPI_Wtime. За время опыта принимался максимум локальных времён, полученный через MPI_Reduce с операцией MPI_MAX. Печать в измеряемый участок не входила.

Для каждой пары (p, n) выполнен один прогревочный и пять измерительных запусков; итоговым значением считалась медиана. Использованы $p = 1, 2, 4, 8, 16$. Измерения проводились на Intel Core i5-12400F (шесть ядер, 12 аппаратных потоков) под Linux 7.3.0-4-cachyos с Open MPI 5.0.11. Программа собиралась GCC 16.2.1 в режиме Release. При $p = 16$ использован параметр --oversubscribe, поэтому этот режим отмечен как переподписка.

6.2. Результаты неупорядоченного приёма

Сводный вывод сценария измерений представлен на рисунке 7, а медианы переведены в микросекунды и приведены в таблице 2.

```
❯ column -s, -t pr1/results/task1_benchmark_summary_20261029_212924.csv
task   processes  elements  bytes   repetitions  median_seconds
task1  1          1         4       5            0.000000123
task1  1          1024      4096    5            0.000000122
task1  1          65536     262144  5            0.000000148
task1  2          1         4       5            0.000015996
task1  2          1024      4096    5            0.000023939
task1  2          65536     262144  5            0.000100111
task1  4          1         4       5            0.000019967
task1  4          1024      4096    5            0.000030335
task1  4          65536     262144  5            0.000155533
task1  8          1         4       5            0.000089125
task1  8          1024      4096    5            0.000074976
task1  8          65536     262144  5            0.000608820
task1  16         1         4       5            0.000123590
task1  16         1024      4096    5            0.000218090
task1  16         65536     262144  5            0.000648993
```

Рисунок 7 – Сводные результаты измерений для задания 1

Таблица 2 – Медианное время неупорядоченного обмена

p	$T(n = 1)$, мкс	$T(n = 1024)$, мкс	$T(n = 65536)$, мкс
1	0,123	0,122	0,148
2	15,996	23,939	100,111
4	19,967	30,335	155,533
8	89,125	74,976	608,820
16	123,590	218,090	648,993

На рисунке 8 показана зависимость времени от числа процессов. Контрольные точки $p = 1$ не включены, поскольку обмен в них отсутствует.

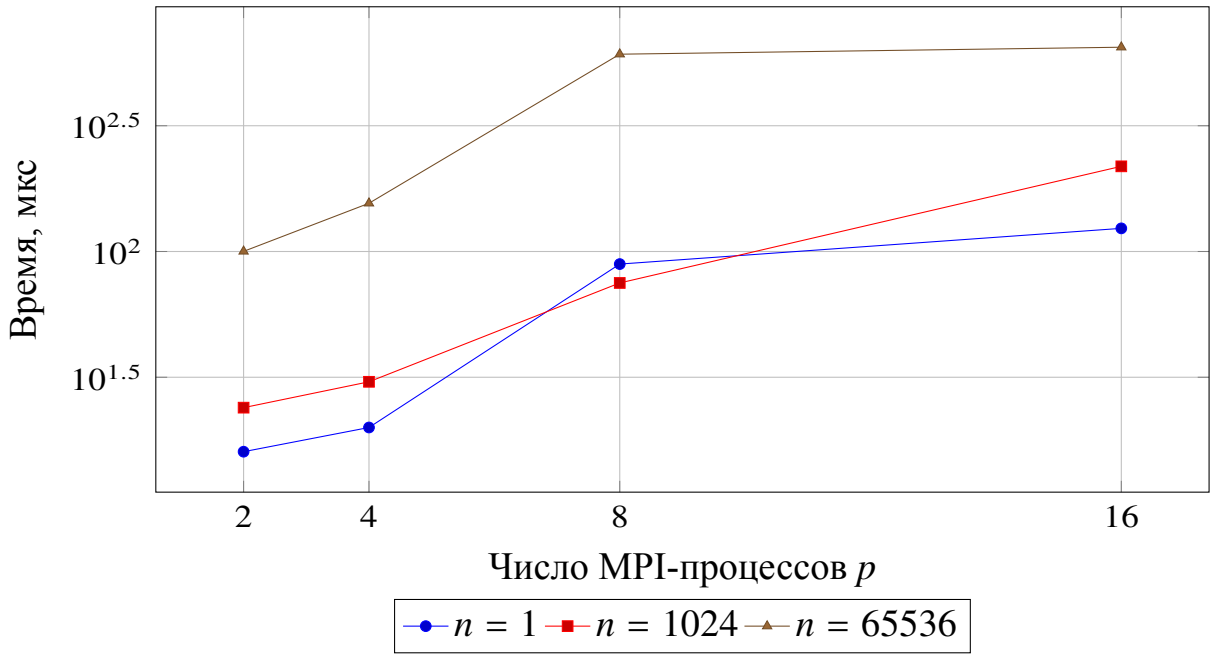


Рисунок 8 – Зависимость времени неупорядоченного обмена от числа процессов

Влияние размера сообщения показано на рисунке 9. Для больших буферов рост времени выражен сильнее, чем для единичного сообщения.

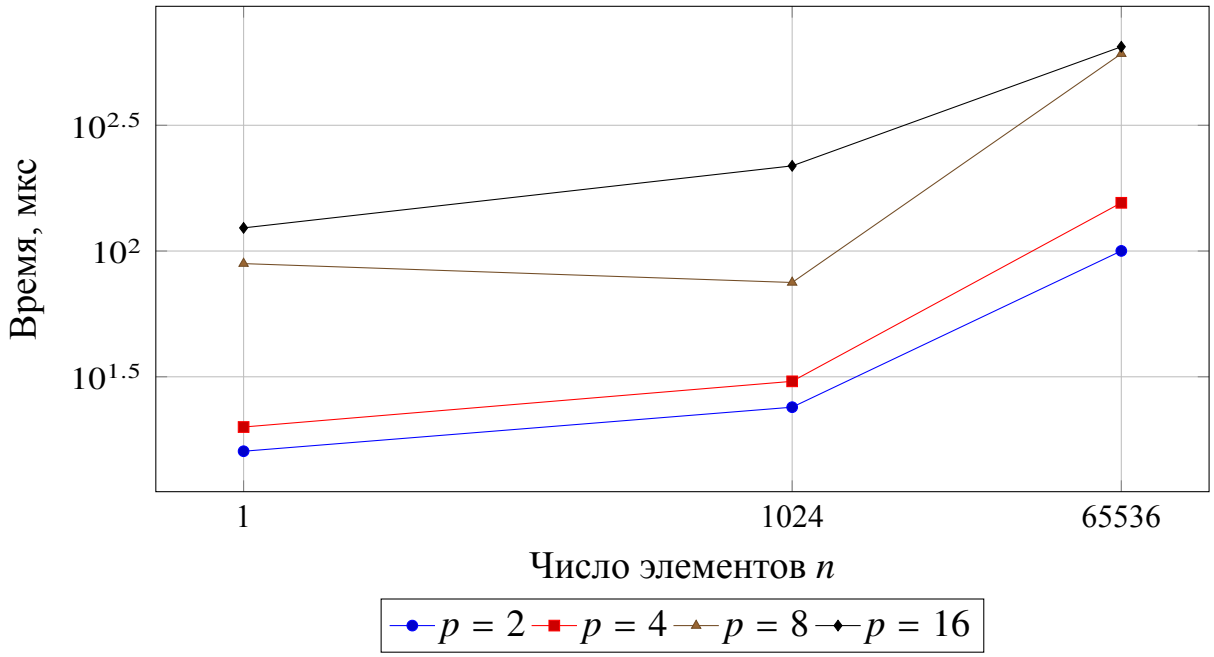


Рисунок 9 – Зависимость времени неупорядоченного обмена от размера сообщения

При $p = 2$ увеличение n от 1 до 65536 привело к росту медианы с 15,996 до 100,111 мкс. Для $p = 8$ и $n = 65536$ получено 608,820 мкс, что примерно в

шесть раз больше значения при $p = 2$. Это согласуется с тем, что при фиксированном n суммарный объём равен $(p - 1)n$ целых чисел. Немонотонность для малых сообщений, в частности 89,125 мкс при $p = 8$, $n = 1$ и 74,976 мкс при $n = 1024$, объясняется сопоставимыми с полезной работой накладными расходами и вариативностью планирования. Точки $p = 16$ дополнительно искажены переподпиской.

6.3. Ограничения интерпретации

При $p = 1$ отправители отсутствуют, поэтому полученные значения около 0,12 мкс отражают лишь накладные расходы таймера. Отношение $T(1, n)/T(p, n)$ не является ускорением: при росте p число отправителей и общий объём передачи увеличиваются. Поэтому графики ускорения и эффективности для данной постановки не строились.

На текущем этапе измерена только версия с MPI_ANY_SOURCE. Для строгого сравнения необходимо провести ту же серию опытов для упорядоченного приёма при тех же (p, n); это следующий этап работы.

7 Выводы

В ходе работы реализованы и проверены два способа приёма MPI-сообщений. Источник MPI_ANY_SOURCE не задаёт порядок вывода: в пяти запусках на восьми процессах получены различные последовательности. Явный источник i обеспечил возрастающий порядок $0, 1, \ldots, p - 1$ во всех функциональных запусках.

Для неупорядоченного приёма выполнено измерение времени для трёх размеров сообщения и $p = 1, 2, 4, 8, 16$. При росте размера буфера и числа отправителей время в целом увеличивалось. Для $n = 65536$ оно возросло с 100,111 мкс при $p = 2$ до 608,820 мкс при $p = 8$. Режим $p = 16$ выполнялся с переподпиской, поэтому его значения нельзя напрямую сравнивать с режимами без неё.

Полученные времена не описывают ускорение, так как при увеличении p одновременно растёт общий объём передачи, а при $p = 1$ обмен отсутствует. Следующий этап --- измерить упорядоченную версию по той же методике и сравнить алгоритмы при равных (p, n).

8 Список использованных источников

1 Мараховский В. Б., Розенблюм Л. Я., Яковлев А. В. Моделирование параллельных процессов. Сети Петри. СПб.: Профессиональная литература, 2014.

ПРИЛОЖЕНИЕ А

Листинг программы с неупорядоченным приёмом

```
// Автор: Марк Пархоменко
// Дата: 27.10.2026

#include <stdio.h>
#include <mpi.h>

int main(int argc, char **argv)
{
   int rank;
   int process_count;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &process_count);

   if (rank == 0)
   {
      printf("rank: %d\n", rank);

      for (int i = 1; i < process_count; i++)
      {

         int received_rank;
         MPI_Status status;
```

```
        MPI_Recv(
            &received_rank,
            1,
            MPI_INT,
            MPI_ANY_SOURCE,
            0,
            MPI_COMM_WORLD,
            &status
        );

        printf("Received rank %d, Sender %d\n",
            received_rank,
            status.MPI_SOURCE);
    }
}

else
{
    MPI_Send(
        &rank,
        1,
        MPI_INT,
        0,
        0,
        MPI_COMM_WORLD
    );
}
```

```
    MPI_Finalize();

    return 0;
}
```

ПРИЛОЖЕНИЕ Б

Листинг программы с упорядоченным приёмом

```
// Автор: Марк Пархоменко
// Дата: 27.10.2026

#include <stdio.h>
#include <mpi.h>

int main(int argc, char **argv)
{
   int rank;
   int process_count;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &process_count);

   if (rank == 0)
   {
      printf("rank: %d\n", rank);

      for (int i = 1; i < process_count; i++)
      {

         int received_rank;
         MPI_Status status;
```

```
        MPI_Recv(
            &received_rank,
            1,
            MPI_INT,
            i,
            0,
            MPI_COMM_WORLD,
            &status
        );

        printf("Received rank %d, Sender %d\n",
            received_rank,
            status.MPI_SOURCE);
    }
}

else
{
    MPI_Send(
        &rank,
        1,
        MPI_INT,
        0,
        0,
        MPI_COMM_WORLD
    );
}
```

```
    MPI_Finalize();

    return 0;
}
```

ПРИЛОЖЕНИЕ В

Листинг программы измерения времени

```
// Автор: Марк Пархоменко
// Дата: 27.10.2026

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char **argv)
{
   int rank;
   int process_count;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &process_count);

   if(argc != 2 )
   {
      if (rank == 0)
         printf("Wrong number of argc\n");
      MPI_Finalize();
      return 1;
   }

   int n = atoi(argv[1]);
```

```
if (n <= 0)
{
   if(rank == 0)
   {
      printf("Buffer size must be positive\n");
   }
   MPI_Finalize();
   return 1;
}

int *buffer = malloc((size_t)n * sizeof(*buffer));
if (!buffer)
{
   printf("Failed to allocate buffer\n");
   MPI_Abort(MPI_COMM_WORLD, 1);
   return 1;
}

if (rank != 0)
{
   for (int i = 0; i < n; i++)
   {
      buffer[i] = rank;
   }
}

MPI_Barrier(MPI_COMM_WORLD);
```

```
double  start_time = MPI_Wtime();
if (rank != 0)
{

   MPI_Send(
      buffer,
      n,
      MPI_INT,
      0,
      0,
      MPI_COMM_WORLD
   );
}
else
{
   for (int i = 1; i < process_count; i++)
   {
      MPI_Status status;
      MPI_Recv(
         buffer,
         n,
         MPI_INT,
         MPI_ANY_SOURCE,
         0,
         MPI_COMM_WORLD,
         &status
      );
   }
```

```
    }

    double elapsed = MPI_Wtime() − start_time;
    double max_elapsed = 0.0;

    MPI_Reduce(
        &elapsed,
        &max_elapsed,
        1,
        MPI_DOUBLE,
        MPI_MAX,
        0,
        MPI_COMM_WORLD
    );

    if (rank == 0)
    {
        printf("processes=%d size=%d time=%.9f\n",
            process_count,
            n,
            max_elapsed);
    }

    free(buffer);
    MPI_Finalize();

    return 0;
}
```